

React

```
import axios from "axios";
import React, { useEffect, useState } from "react";
import { api } from "../api";

const Axios = () => {
  const [data, setData] = useState([]);
  const fetchData = async () => {
    const result = await axios.get("https://dummyjson.com/products");

    console.log(result.data.products);

    setData(result.data.products);
  };

  const fetchUsers = async () => {
    const result = await api.delete("/users/1");

    console.log(result);
  };

  useEffect(() => {
    fetchData();
    fetchUsers();
  }, []);

  return (
    <div>
      <h1>Axios</h1>

      { /* {data.map((ele) => {
        return (
          <div
            key={ele.id}
            style={{ border: "2px solid yellow", padding: "10px" }}
          >
            <h1>{ele.title}</h1>
            <img src={ele.images[0]} alt="" />
          </div>
        );
      }}} */ }
    </div>
  );
};

export default Axios;
```

```
import axios from "axios";

export const api = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com",
});
```

Theory

- **Axios Basics**
 - Promise-based HTTP client for browser and Node.js.
 - Common methods:
 - `axios.get(url)` → fetch data.
 - `axios.post(url, body)` → send data.
 - `axios.put(url, body)` → update data.
 - `axios.delete(url)` → delete data.
- **Axios Instance**
 - `axios.create({ baseURL })` → creates a reusable instance with a base URL.
 - Useful for APIs with multiple endpoints.
- **useEffect with Axios**
 - API calls are side effects → handled inside `useEffect`.
 - Dependency array `([])` ensures the call runs only once on mount.
- **State Management**
 - `useState` stores API response data.
 - `setData(result.data)` updates state after fetching.

Coding Round Questions

- **Axios GET** → Write a React component that fetches product data from an API and displays it in a list.
- **Axios POST** → Implement a form that submits user data to an API using `axios.post`.
- **Axios DELETE** → Write a function that deletes a user by ID using `axios.delete`.
- **Axios instance** → Create an Axios instance with a base URL and use it for multiple API calls.
- **Error handling** → Demonstrate how to handle API errors using `try-catch` or `.catch()`.

- Loading state → Add a loading spinner while Axios fetches data.

⚡ Machine Coding Round Challenge

Problem: User Management Dashboard with Axios

Requirements:

1. Create an Axios instance with base URL `https://jsonplaceholder.typicode.com`.
2. Implement CRUD operations:
 - GET /users → fetch and display all users.
 - POST /users → add a new user via form.
 - PUT /users/:id → update user details.
 - DELETE /users/:id → delete a user.
3. Use `useState` to store users and `useEffect` to fetch on mount.
4. Display users in a styled list with edit/delete buttons.
5. Show loading state while fetching and error messages if API fails.

Expected Flow:

- On mount → fetch users and display them.
- Add user → form submits via `axios.post`.
- Edit user → update via `axios.put`.
- Delete user → remove via `axios.delete`.
- UI updates dynamically after each action.